Towards Understanding the Bugs in Verilator, a Hardware Description Language Compiler

SONGYAN JIANG*, Nanjing University, China
MAOLIN SUN*, Nanjing University, China
KANG CHEN, Nanjing University, China
QINGYANG LI, Nanjing University, China
YIBIAO YANG†, Nanjing University, China
YUMING ZHOU, Nanjing University, China

Verilator is the premier open-source Hardware Description Language (HDL) compiler. It transforms Verilog and SystemVerilog designs into optimized C++ or SystemC models, enabling high-speed, cycle-accurate simulation prior to large-scale production. As a cornerstone of the hardware verification ecosystem, the correctness of Verilator is paramount; compiler faults can lead to silent simulation errors or unexpected failures, undermining the integrity of the hardware development lifecycle. Unlike traditional software compilers, HDL compilers manage unique concurrency and synthesis semantics, potentially introducing distinct bug patterns and complexities. However, while prior research has explored testing techniques for HDL toolchains, there remains a lack of systematic empirical studies characterizing the specific nature of bugs in Verilator. This knowledge gap hinders the development of targeted improvements in compiler robustness and testing strategies. To address this, we present the first comprehensive empirical study of Verilator bugs. We manually collected, analyzed, and categorized a dataset of 488 confirmed bugs from the official repository over three years. Our study investigates bug symptoms, root causes, and the characteristics of triggering test cases, while also evaluating the effectiveness of existing testing techniques. Based on our findings, we provide actionable guidance for developers to enhance Verilator's reliability and for researchers to design more effective automated testing methodologies for HDL compilers.

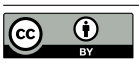
CCS Concepts: • **Hardware → Hardware description languages and compilation**; **Software tools for EDA**; • **Software and its engineering → Software testing and debugging**.

Additional Key Words and Phrases: Verilog compiler, empirical study, bug

ACM Reference Format:
Songyan Jiang, Maolin Sun, Kang Chen, Qingyang Li, Yibiao Yang, and Yuming Zhou. 2026. Towards Understanding the Bugs in Verilator, a Hardware Description Language Compiler. *Proc. ACM Softw. Eng.* 3, ISSTA, Article ISSTA135 (October 2026), 22 pages. https://doi.org/10.1145/3832226

*Both authors contributed equally.
†Corresponding author.

Authors' Contact Information: Songyan Jiang, Nanjing University, State Key Laboratory for Novel Software Technology, Nanjing, China, songyanjiang@smail.nju.edu.cn; Maolin Sun, Nanjing University, State Key Laboratory for Novel Software Technology, Nanjing, China, merlin@smail.nju.edu.cn; Kang Chen, Nanjing University, State Key Laboratory for Novel Software Technology, Nanjing, China, k.chen@smail.nju.edu.cn; Qingyang Li, Nanjing University, State Key Laboratory for Novel Software Technology, Nanjing, China, liqingyang@smail.nju.edu.cn; Yibiao Yang, Nanjing University, State Key Laboratory for Novel Software Technology, Nanjing, China, yangyibiao@nju.edu.cn; Yuming Zhou, Nanjing University, State Key Laboratory for Novel Software Technology, Nanjing, China, zhouyuming@nju.edu.cn.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2026 Copyright held by the owner/author(s).
ACM 2994-970X/2026/10-ARTISSTA135
https://doi.org/10.1145/3832226

1 Introduction

Simulation is fundamental to modern hardware development, as hardware designs must undergo rigorous simulation to validate their functionality and correctness prior to large-scale production. Given the growing complexity of hardware systems, reliable simulation is essential for identifying defects early and preventing costly downstream failures [30]. Hardware designs are typically specified using Verilog, a standard Hardware Description Language (HDL), or its extension, SystemVerilog [18], which offers advanced modeling capabilities. To support the validation of such designs, the hardware industry relies on a rich ecosystem of simulation and analysis tools.

Among these, Verilator [23] has emerged as a leading open-source HDL compiler that generates high-performance, cycle-accurate simulators, and it is widely adopted in both academic research and industrial practice [13]. Verilator's prominence stems from an architectural approach that fundamentally differs from traditional simulators like Icarus Verilog [31]. Rather than interpreting HDL constructs, Verilator compiles Verilog and SystemVerilog specifications into highly optimized C++ or SystemC implementations. By translating hardware logic into executable software models, it achieves substantially higher simulation performance than interpretation-based alternatives [2] and is regarded as the *fastest* open-source simulator available [27]. Owing to its efficiency, Verilator is frequently used as a reference execution engine [35] and has become the preferred tool for computationally intensive validation tasks, including large-scale hardware fuzzing and architectural exploration [8].

Like modern software compilers, Verilator employs sophisticated program analysis and optimization passes to achieve high performance. Instead of translating HDL constructs into C++ through a naive one-to-one mapping, Verilator performs aggressive optimization to generate highly efficient simulation models [28]. These models support parallel execution and are exposed via C++ interfaces, enabling seamless integration with software-based testbenches. Consequently, Verilator often delivers performance comparable to commercial simulators [24, 25] and significantly superior to interpreted open-source alternatives. This capability is indispensable for pre-silicon verification and hardware–software co-simulation, where validating complex designs at scale would otherwise be infeasible [6].

However, Verilator's performance-oriented compilation pipeline inherently increases system complexity and the potential for implementation defects. To maximize execution speed, the compiler occasionally deviates from strict HDL semantics. Notably, most of these deviations are intentional, well-documented, and benign in practice. For example, conventional hardware simulators use a four-state logic model (`0`, `1`, `X` for unknown, and `Z` for high impedance) to accurately represent physical circuits. To accelerate simulation, Verilator defaults to a simplified two-state boolean model (`0` and `1`), coercing any `X` or `Z` states into standard binary values. Developers typically mitigate this limitation by using static analysis and initialization flags (e.g., `-x-assign`)[1] to ensure their designs do not rely on these hardware-specific states. Furthermore, when Verilator encounters constructs that disrupt its optimized execution model, it safely defaults to issuing conservative warnings that halt compilation. Nevertheless, some performance-driven optimizations can introduce subtle, hard-to-detect behavioral inconsistencies. Consider a scheduling race condition where two procedural blocks concurrently read and write to a shared variable. A standard event-driven simulator, such as Icarus Verilog [31], resolves this non-deterministically at runtime based on its internal event queue. In contrast, Verilator statically schedules these blocks at compile time to optimize performance, enforcing a fixed execution order. If this statically chosen order coincidentally matches the developer's intent, the simulation will produce the expected output. This effectively

[1] https://verilator.org/guide/latest/exe_verilator.html#cmdoption-x-assign

masks the underlying semantic deviation, creating a false sense of correctness that may fail under different conditions or downstream tools.

Systematically identifying and characterizing these issues is challenging, especially in the absence of reliable execution oracles or semantics-equivalent reference implementations. Despite their critical role in hardware verification, the reliability of Verilog compilers has received relatively limited attention in the software engineering research community. Existing research remains sparse, and the primary prior work focuses on discovering individual bugs rather than providing a systematic, empirical characterization of real-world reliability issues in Verilog compilers [35]. Moreover, Verilog compilers pose challenges that differ substantially from those of traditional software compilers [5], due to the concurrency-oriented semantics of HDLs and the domain-specific optimizations employed in hardware simulation [26]. These factors collectively motivate a comprehensive empirical investigation of Verilator, with the goal of understanding the types of bugs that arise in its compilation pipeline and how they affect reliability in practice. By analyzing real-world bug reports from Verilator, we aim to uncover systematic reliability trends and identify opportunities to strengthen HDL compiler robustness.

Our work. Motivated by the above observations, this paper presents a systematic empirical study of Verilator. We focus on Verilator due to its broad adoption in both academic research and industrial practice, as well as its open-source nature, which enables detailed inspection of its development history and bug-fixing processes. We do not consider commercial Verilog compilers in this study, as their closed-source designs prevent comprehensive analysis. In addition, we exclude Icarus Verilog [32], another popular open-source tool, because it primarily adopts an interpretation-based simulation model and is mainly designed for educational use rather than large-scale compilation and optimization. To structure our investigation, we formulate the following research questions:

- **RQ1 (Bug Symptoms):** How do Verilator bugs manifest in practice?
- **RQ2 (Root Causes):** What are the common root causes of Verilator bugs?
- **RQ3 (Bug Detection):** How are Verilator bugs typically detected and diagnosed?
- **RQ4 (Existing Techniques):** What techniques can be used to test Verilog compilers?

These questions aim to provide a comprehensive characterization of bugs within Verilator. We begin with *RQ1* and *RQ2*, which examine where Verilator bugs originate and how they manifest during compilation and simulation. By correlating root causes with symptoms, we aim to construct a holistic view of the reliability challenges inherent in Verilator's compilation pipeline. To support this analysis, we curated and analyzed a dataset of 488 valid bug reports and their corresponding fixes from the total 1,390 issues, collected from Verilator's official GitHub repository[2] over a three-year period. We manually classified these bugs to establish a taxonomy, labeling each issue according to both its symptom and underlying cause. Next, *RQ3* investigates the characteristics of the Verilog code required to trigger these bugs. This analysis isolates practical bug revealing scenarios and offers insight into the semantic and structural complexity of bug-triggering inputs. Finally, *RQ4* bridges our empirical findings with the current state-of-the-art testing techniques for Verilog compilers, assessing their ability to expose bugs in Verilator. Based on this evaluation, we discuss the strengths and limitations of existing approaches and identify gaps where additional robustness is needed. Collectively, these analyses provide a systematic understanding of Verilator's reliability and offer actionable insights for researchers and practitioners aiming to advance the development and testing of Verilator.

[2] https://github.com/verilator/verilator

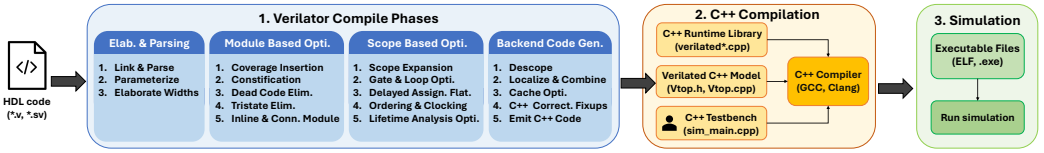


Fig. 1. An overview of the Verilator workflow.

Contributions. Our main contributions are as follows:

- We present the first comprehensive empirical study of Verilator, a leading open-source Hardware Description Language (HDL) compiler. We systematically analyze real-world bugs in Verilator, characterizing their symptoms, root causes, fix complexity, and detection mechanisms.
- We construct a taxonomy of bugs within the Verilator compilation pipeline. This analysis reveals how specific compilation stages contribute to bug introduction and persistence, offering new insights into the inherent complexity of Verilog compilation.
- We examine existing testing techniques for Verilog compilers, assessing their effectiveness in uncovering bugs in Verilator. Building on our analysis, we provide actionable suggestions for developers and researchers to improve the robustness of HDL compilers.

Findings. Several key findings emerge from our study:

- ⋆ **Prevalence of Silent Failures:** *Miscompilation* is the second largest bug symptom (25.20%). Unlike explicit crashes, these silent failures allow the compiler to terminate successfully while producing functionally incorrect logic, posing a significant risk to hardware integrity.
- ⋆ **Architectural Dichotomy of Root Causes:** We identify a clear divide between compiler stages and bug manifestations. While frontend issues in *Elaboration and Semantics* primarily drive system instability (crashes), backend complexities in *Scheduling and Emission* account for 75% of invalid code generation.
- ⋆ **Complexity in Semantic Density:** Bug-triggering test cases are typically small (median 15 LOC) but semantically dense. Failures are rarely caused by deep nesting; instead, they are driven by complex interactions between high-level software-like abstractions, such as *Classes* and *Structs*, and legacy hardware constructs.
- ⋆ **Stagnation of Existing Testing Tools:** An evaluation of state-of-the-art fuzzers reveals a significant *semantic gap*. Existing tools focus on syntactic validity but fail to generate the semantic complexity required to trigger sophisticated optimization passes or exercise advanced SystemVerilog features, resulting in minimal bug discovery in modern releases.

2 Background

Verilator is an open-source, high-performance simulation system widely used for Verilog and SystemVerilog designs. By enabling rigorous functional verification prior to physical fabrication, it plays a critical role in identifying design defects that could otherwise lead to catastrophic hardware failures or substantial financial losses. Unlike traditional interpreted simulators, Verilator employs an ahead-of-time compilation strategy. Its primary function is to transform HDL code into equivalent C++ or SystemC models, which are subsequently compiled into native executable binaries. Figure 1 outlines this workflow. Specifically, the process begins with *Elaboration and Parsing*. During this phase, *parsing* converts the HDL source into an abstract syntax tree (AST), while *linking* resolves module instantiations and signal references across files to construct a cohesive design hierarchy. Verilator then propagates parameters and elaborates data types and expression widths to fully resolve the design. Following elaboration, Verilator applies extensive module- and scope-based

optimizations (e.g., dead code elimination, flattening) rather than performing a naive, one-to-one HDL-to-C++ translation. By eliminating interpretation overhead and aggressively optimizing code, Verilator achieves performance comparable to commercial tools and frequently runs hundreds of times faster than interpreted simulators like Icarus Verilog [27].

Against this technical backdrop, this study is driven by the unique intersection of Verilator's architectural complexity, its transparent development model, and its critical role in hardware verification. First, because Verilator operates as a compilation system, handling parsing, semantic analysis, optimization, and target-specific code generation, it is susceptible to similar class of subtle defects found in software compilers. Second, Verilator's community-driven development on GitHub provides a transparent, traceable history of bug reports, discussions, and resolutions, offering a rich dataset for empirical analysis. Finally, given Verilator's widespread adoption in both industrial and academic workflows for tasks ranging from architectural exploration to hardware–software co-design, understanding its defect patterns is essential. Thus, a systematic examination of these bugs not only aids in hardening the system itself but also contributes to the broader body of knowledge regarding quality assurance for HDL simulators.

3 Study Methodology

This section describes how we collected, curated, and analyzed Verilator bugs. Figure 2 summarizes our overall workflow, which consists of two main phases. First, we systematically collected reproducible bug reports from Verilator's official GitHub repository.[3] Second, we manually analyzed these bugs along multiple dimensions to enable repeatable quantitative and qualitative analyses addressing RQ1–RQ4.

3.1 Collecting Bugs and Fixes

Our study targets the official Verilator GitHub repository (`verilator/verilator`), including 1,390 closed issues created between October 30, 2022 and October 30, 2025. This window started from a major new release v5.002[4] (October 29, 2022) which introduced substantial internal changes, and ended before the release of v5.042[5] (November 2, 2025). Since the developers ask users to report bugs against the latest version, we consider all issues reported to focus on the v5.x series. By starting immediately after the release v5.002, we focus on bugs reported under a relatively stable architectural baseline while avoiding transitional noise from pre-release development. Overall, this window spans the transition from v4.x to v5.x and covers several years of active development, making it well suited for studying both persistent and newly introduced issues. We only consider closed issues because they have been fully processed by developers and typically include resolution labels, discussion threads, and fixing commits. Notably, because Verilator manages issues and pull requests within the same repository, no additional data sources or cross-repository crawling are required.

To conduct this study, we begin with all 1,390 closed issues reported during the selected time window. From this pool, we exclude issues explicitly labeled by developers with resolutions indicating that no actionable defect exists. Specifically, as illustrated in `Step 1: Label Exclusion` of Figure 2, we excluded issues that developers resolved as invalid, external to the project, or abandoned. We further refined this by removing any issues with a closing status of `Not planned` or `Duplicate`. Additionally, we filtered issues based on specific operational labels. Reports tagged with `status: asked reporter`, `status: discussion`, or `status: blocked` were removed from

[3] https://github.com/verilator/verilator/issues
[4] https://github.com/verilator/verilator-announce/issues/57
[5] https://github.com/verilator/verilator-announce/issues/80

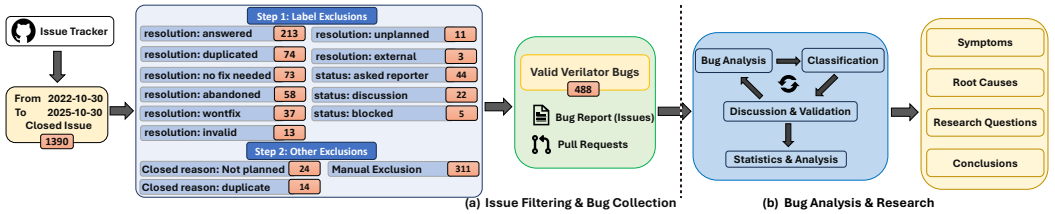


Fig. 2. The overview of our bug collection and analysis process.

our dataset. These labels typically indicate community inquiries, incomplete reports awaiting user feedback, or issues contingent upon the resolution of other prerequisites, rather than independent software defects. For `Step 2: Other Exclusions`, we further excluded the issues based on the developers' terminal closing reasons, including any reports marked as `Not planned` or `Duplicate` at the time of closure. The exclusions automatically remove reports that do not reflect real and fixable problems in Verilator and reduce the candidate set to 799 issues.

Nonetheless, automatic labels alone are insufficient to reliably distinguish all real bugs from non-actionable reports. Therefore, two authors manually inspected each candidate issue together with its linked pull requests and discussion threads. We excluded issues that fall into the following categories: duplicate reports already identified and commented by developers; intended behavior rather than defects; issues later declared irreproducible; discussion-only threads such as questions or feature requests; and issues related solely to external infrastructure (e.g., build systems or continuous integration). Furthermore, we also verified that all retained issues originated within Verilator itself rather than in downstream toolchains. Particularly, because Verilator's execution model involves translating HDL into C++ and relying on external compilers to generate the final binary, failures can sometimes stem from the downstream compiler. To maintain the internal validity of our study, we strictly excluded issues where Verilator generated correct C++ code but the downstream compiler failed to process it as expected.[6] After this manual inspection, 488 issues were identified as **valid** Verilator bugs, while the rest were classified as other non-bug issues. Each retained bug is required to provide concrete technical artifacts that enable analysis and, where possible, reproduction. Specifically, every bug typically includes:
(a) a Verilog or SystemVerilog test case, either provided by developers or reduced by us;
(b) a corresponding fixing commit, linked via a pull request or direct repository commit; and
(c) a failure oracle, such as an assertion backtrace for compiler crashes or a reference output for miscompilation bugs.
These artifacts serve as the foundation for the empirical analysis.

3.2 Analyzing Bugs

To answer RQ1–RQ4, we systematically analyze the 488 collected valid bugs in Verilator. Following prior bug analysis approaches [4] and empirical studies of compiler bugs [15, 29, 36], we adopt a structured, multi-dimensional coding scheme that captures both internal implementation causes and external user-visible effects with reference to Qualitative Content Analysis (QCA) [20]. Specifically, each bug is initially analyzed along two orthogonal dimensions. **Symptom** describes how the bug manifests externally, including compiler crashes, incorrect simulation results, misleading diagnostics, or unintended rejection or acceptance of input programs. **Root cause** characterizes the underlying implementation error responsible for the defect and is determined primarily

[6] https://github.com/verilator/verilator/issues/5754 is an example that was initially reported as a Verilator defect but was ultimately traced to a bug in g++.

Table 1. Classification of Bug Symptoms in Verilator

Symptom	Description	# Bugs
Crash	Unexpected termination of the compiler (e.g., Segfault, Internal Error, Assertion Failure) preventing output generation. Includes: • *Internal Error (115):* Assertion/internal rule violation with diagnostic abort. • *Memory Error (21):* Invalid memory operation terminated by OS or ASAN. • *Other Crashes (10):* Miscellaneous issues not fitting the above subcategories.	146
Miscompilation	The compiler completes, but the output is defective. Includes: • *Behavioral Divergence (109):* Simulation runs but violates HDL semantics. • *Runtime Error (14):* Simulation builds but crashes/aborts during execution.	123
Language Compliance	Failures regarding adherence to the language standard: • *Reject-Valid (102):* Fails to compile a valid design (False Positive). • *Accept-Invalid (16):* Silently accepts an ill-formed design (False Negative).	118
Invalid C++	Generated code fails to compile with downstream C++ compilers (e.g., g++/clang).	64
Diagnostic Issue	Issues with error reporting accuracy: • *Omission (3):* Fails to report a required warning or error. • *Misleading/Inaccurate (14):* Reports the wrong cause or location.	17
Performance	Excessive resource consumption (time/memory) during compilation or significantly degraded simulation runtime speed.	15
Hang	Compiler or generated model enters an infinite loop or deadlock state, requiring manual termination.	5

through inspection of the fixing commit and surrounding code. Together, these dimensions provide complementary views of Verilator bugs from both developer and user perspectives.

To ensure rigorous and consistent labeling, we employed an iterative open coding process to refine our classification scheme [21]. We conducted 10 rounds of annotation, with each round involving 40 randomly selected bugs, which is consistent with prior studies [15]. In each round, two authors independently labeled the bugs, subsequently resolving disagreements through discussion to refine category definitions. Unresolved cases were adjudicated by one additional author. We monitored inter-rater reliability throughout this process. Although the initial agreement was approximately 0.3, these early discrepancies guided necessary refinements to the coding frame. The process converged when both the schema definitions and annotator agreement stabilized. Ultimately, we achieved strong inter-rater reliability on the final dataset, yielding Cohen's κ coefficients exceeding 0.9 across both dimensions. During classification, we observed that a small subset of bugs involved multiple root causes. To prevent double-counting and maintain statistical clarity, we assigned a single primary root cause to these cases, consistent with prior empirical studies [15]. Once the taxonomy was finalized and validated, the remaining bugs were labeled accordingly. This effort produced a meticulously curated dataset of 488 closed, patched Verilator bugs, each featuring high-confidence annotations and reproducible test cases. Specifically, these test cases facilitate the quantitative and qualitative analysis conducted in RQ3. Ultimately, this comprehensive dataset serves as the empirical foundation for all subsequent analyses.

4 RQ1: Bug Symptom

Bug symptoms capture the externally observable behavior of Verilator when a fault is triggered. They are typically manifested through compiler error messages, abnormal termination, unexpected simulation behavior, or deviations from the expected handling of Verilog and SystemVerilog constructs. A clear symptom classification helps characterize how bugs affect users in practice and provides a foundation for understanding their impact on tool reliability and usability. Based on a

```
1  module a(i);
2    function integer f;
3    endfunction
4    input [f()-1:0] i;
5  endmodule
```

(a) A minimal Verilog example that triggers a crash in Verilator.

```
$ verilator --binary a.v
%Error: Internal Error: a.v:5:13: ../V3Number.cpp:1005:
toUInt with 4-state 32'bxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
                                    : ... note: In instance 'a'
    5 |   input [f()-1:0] i;
      |             ^
```

(b) The crash message produced by Verilator when compiling the code shown in Figure 3a.

Fig. 3. An example of a Verilator crash due to an internal error triggered by a specific Verilog construct. #6534

detailed examination of issue descriptions, diagnostic logs, and reproducing examples for all 488 valid bugs, we classify Verilator bug symptoms into six primary categories, shown in Table 1.

Symptom 1: Crash. A crash refers to unexpected termination of the Verilator compilation process, preventing the generation of any output. These failures often manifest as segmentation faults or `Internal Error` messages emitted during compilation. Crashes typically indicate unhandled corner cases within the compiler and halt the toolchain entirely. We identify 146 bugs (29.92%) that exhibit crash behavior, making this the most prevalent symptom category. We further divide this category into three subtypes.

- *Internal Error:* The compiler explicitly aborts the compilation process upon detecting a violation of its internal rules or failing an assertion, typically emitting an internal diagnostic message (e.g., `%Error: Internal Error: [error message]`).
- *Memory Error:* The compiler process terminates due to an invalid memory operation, such as a null-pointer dereference, out-of-bounds access, or use-after-free. Unlike internal errors, these failures trigger an abnormal termination enforced by the operating system (e.g., a `SIGSEGV`) or are intercepted by memory safety tools like AddressSanitizer (ASAN), bypassing standard compiler diagnostics.
- *Other Crashes:* Remaining bugs that do not fit neatly into the above two subcategories. These bugs were typically reported without code snippets or error logs, such as Issue#3933.

Figure 3 illustrates an example of a crash. In this example, a function shown in Figure 3a without a return path is used in a constant expression that determines a signal width. During constant evaluation, Verilator attempts to convert an undefined four-state logic value into a concrete integer, triggering an internal error in the numeric backend. The compiler terminates abruptly with an error message indicating an illegal conversion. Developers resolved this issue by adding a check to detect functions that lack return statements and emit a `NORETURN` diagnostic, preventing undefined values from reaching later compilation stages.

Symptom 2: Miscompilation. Miscompilation refers to cases where Verilator and its downstream C++ compiler complete compilation, but the generated output is defective. These bugs are particularly harmful because the compiler appears to succeed without any error, yet produces incorrect or unusable results. We identify 123 bugs (25.20%) in this category and further divide it into two subtypes.

- *Behavioral Divergence* cases generate compilable and executable simulations, but the runtime behavior deviates from the expected HDL semantics or from reference simulators. Figure 4 presents a minimal example where a constant left-shift operation with a large shift amount produces an incorrect simulation result. The error arises from an integer overflow in the internal loop indices used during constant evaluation. To address this issue, developers added explicit checks for invalid bit ranges where the least significant bit exceeds the most significant bit, allowing Verilator to correctly resolve the expression to zero.

```
module a;
  wire [5:0] out = 6'o66 <<< 32'hFFFF_FFFF;
  initial begin
  $display("got=%0d", out); $finish; end
endmodule
```

(a) A minimal Verilog example that produces wrong runtime result in Verilator.

```
$ verilator --binary a.v
$ ./obj_dir/Va
got=27
a.v:5: Verilog $finish
```

(b) The wrong runtime result produced by Verilator when running the above code.

Fig. 4. An example of a Verilator wrong runtime result. #6007

```
module a(input logic signed[64:0] i_x, output logic signed[64:0] o_y);
    struct { logic signed[64:0] x; } s;
    assign s.x = i_x;
    assign o_y = -s.x;
endmodule
```

(a) A minimal Verilog example that triggers invalid C++ code in Verilator.

```
$ verilator --binary a.v
Va_xx__0.cpp:28:28: error: invalid cast from type 'VlWide<3>' to type 'IData' {aka 'unsigned int'}
   28 |    VL_NEGATE_I(__Vtemp_2, (IData)(vlSelfRef.a__DOT__s.__PVT__x));
      |                            ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
Va_xx__0.cpp:28:5: error: 'VL_NEGATE_I' was not declared in this scope; did you mean 'VL_NEGATE_W'?
   28 |    VL_NEGATE_I(__Vtemp_2, (IData)(vlSelfRef.a__DOT__s.__PVT__x));
      |    ^~~~~~~~~~
      |    VL_NEGATE_W                                              ^
```

(b) The invalid C++ code produced by Verilator when compiling the above code.

Fig. 5. An example of a Verilator invalid C++ code. #6186

- *Runtime Error* miscompilations occur when the generated simulation builds successfully but crashes or aborts during execution. Although less frequent (14 bugs), these cases can be difficult to diagnose because the failure is deferred to runtime and may only appear under specific inputs or execution paths.

Symptom 3: Language Compliance Issues. Language compliance issues arise when Verilator fails to adhere to the Verilog or SystemVerilog language specification. We identify 118 such bugs (24.18%), which fall into two complementary subcategories. Specifically, *Reject-Valid* bugs incorrectly reject valid designs, resulting in false positives that reduce portability and usability. They account for most cases in this category (102 bugs). *Accept-Invalid* bugs silently accept ill-formed designs, allowing erroneous constructs to pass through compilation without diagnostics. Both cases undermine the compiler's role as a standards-conforming implementation and can lead to downstream failures that are difficult to trace.

Symptom 4: Invalid C++. This category encompasses instances where Verilator generates C++ code that fails to compile with standard downstream compilers, such as `g++` or `clang++`. Unlike silent miscompilations, these defects are immediately detectable via the downstream compiler's error output [7]. We identify 64 bugs (13.11%) in this category. Figure 5 shows an example involving a signed SystemVerilog struct field wider than 64 bits. When negating the field, Verilator incorrectly selects a narrow integer negation helper instead of the wide-version operator, producing invalid C++ code with incompatible casts and undefined helper macros. The issue was fixed by refining the type selection logic to preserve width information during code generation.

Symptom 5: Diagnostic Issues. Diagnostic issues concern inaccuracies in error reporting rather than core compilation behavior. We identify 17 bugs (3.48%) in this category. Some involve *Omission*, where required warnings are not reported. Others involve *Misleading or Inaccurate* diagnostics,

where the reported cause or source location does not correspond to the actual fault. Although these bugs do not directly affect correctness, they significantly hinder debugging and user understanding.

Symptom 6: Performance Issues. Performance issues refer to excessive consumption of time or memory during compilation, or substantial slowdowns in the generated simulation models. We identify 15 bugs (3.07%) in this category. While these issues do not necessarily affect functional correctness, they can severely impact usability, especially for large-scale or industrial designs.

Symptom 7: Hangs. A hang occurs when Verilator or the generated simulation model becomes unresponsive due to infinite loops or deadlock conditions. Unlike crashes, hangs do not terminate execution and typically require manual intervention. Although relatively rare, we identify 5 bugs (1.02%) that exhibit hang behavior, which can be especially disruptive in automated testing and continuous integration workflows.

Overall, crashes and miscompilations constitute the most frequent symptoms, together accounting for over 55% of the analyzed cases. This indicates that a large portion of the studied defects either abruptly halt the compilation process or silently produce incorrect simulation behavior. Language compliance issues form the next largest group, reflecting the practical challenges of implementing a complex hardware description language standard. Additionally, although the remaining categories, including diagnostic issues, performance problems, and hangs, occur less frequently, they can still negatively affect downstream users. For instance, precise diagnostics are closely tied to simulation efficiency; Verilator's official documentation[7] notes that resolving a single `UNOPTFLAT` warning can yield up to a 60% improvement in performance. Consequently, even seemingly minor defects, such as missing or inaccurate warnings, can result in missed optimization opportunities or hinder the hardware debugging process.

> **Finding 1:** ***Crashes*** *(29.92%),* ***miscompilations*** *(25.20%), and* ***language compliance issues*** *(24.18%) are the most prevalent bug symptoms, collectively accounting for 79.30% of the analyzed dataset. Notably, miscompilations represent silent failures that yield incorrect simulation behavior despite a successful compilation, posing a severe risk to verification reliability. Furthermore, compliance issues are predominantly "reject-valid" errors. These findings indicate that Verilator's primary challenges lie in maintaining functional correctness and robust front-end language support.*

5 RQ2: Bug Cause

Root causes of bugs capture internal problems in the tool's implementation, such as errors introduced in specific compilation phases or flawed design choices in the pipeline. Understanding these causes allows us to identify recurring weaknesses in Verilator. Our analysis is based on a close examination of each valid bug together with its corresponding fixing pull request or commit. We study pull request descriptions, developer discussions, and code changes to determine the primary factor that led to the failure. This approach enables us to distinguish observable manifestations from their origins in the implementation. Based on this manual investigation, we classify Verilator bugs into nine root cause categories, as summarized in Table 2.

Root Cause 1: Elaboration & Semantics. This category represents the most frequent cause of failure, accounting for 140 bugs. These errors occur during the elaboration phase, where Verilator flattens the module hierarchy, propagates parameters, and resolves the design's internal netlist structure. Failures here typically involve misunderstanding the design context, such as incorrect symbol resolution, parameter evaluation errors, or misinterpretation of language semantics. For example, Figure 6 demonstrates a semantic error caused by incomplete implementation of SystemVerilog structure literals. The code in Figure 6a attempts to initialize a `localparam` using an unpacked structure literal ('{1}). However, the compiler's constant evaluation engine fails to recognize the

[7] https://verilator.org/guide/latest/simulating.html#benchmarking-optimization

Table 2. Taxonomy of Verilator Bug Root Causes.

Root Cause Category	Description	Count
Elaboration & Semantics	Errors in module hierarchy flattening, parameter propagation, and netlist linkage.	140
Scheduling & Emission	Faults in ordering execution logic (breaking combinatorial loops), delay modeling, or C++ output generation.	125
Type System & Widths	Mishandling of bit-width inference, signed/unsigned arithmetic resolution, and data type consistency.	65
Optimization	Incorrect transformation passes such as constant propagation, dead code elimination, or module inlining.	65
Diagnostics & Reporting	Defects in error localization, misleading warning messages, or failure to report valid syntax errors.	24
Syntactic Analysis	Failures in front-end parsing, including Lexer/Bison ambiguities and preprocessor macro handling.	23
I/O & Environment	Faulty file interactions, improper handling of include paths, or issues with OS-specific behavior.	23
Memory Safety	Issues regarding internal memory allocation, pointer management, and object lifetimes within the compiler itself.	19
Others	Miscellaneous issues not fitting the above categories.	4

```
typedef struct {
    int a;
} unpacked_struct;
localparam unpacked_struct PARAM = '{1};
```

(a) A minimal Verilog example triggering an elaboration error.

```
+ } else if (m_params && VN_IS(valuep, ConsPackUOrStruct)) {
+     AstNode* const newp = valuep->cloneTree(false);
+     nodep->replaceWithKeepDType(newp);
+     VL_DO_DANGLING(pushDeletep(nodep), nodep);
+     did = true;
```

(b) The fix patch of the elaboration error.

Fig. 6. An example of a Verilator elaboration and semantics error. #6053

constructor node as a valid compile-time constant, rejecting the valid code. Developers resolved this by explicitly enabling the traversal and resolution of unpacked structure constants during the elaboration phase, as illustrated in Figure 6b. Specifically, when the compiler encounters an unpacked structure (i.e., `ConsPackUOrStruct`) in a parameter context, it now correctly replicates the structure's value and substitutes it into the abstract syntax tree, ensuring the parameter is resolved without error.

Root Cause 2: Scheduling & Emission. The second largest category, comprising 125 bugs, involves faults in the backend processes. Scheduling refers to the ordering of execution logic, such as breaking combinatorial loops and modeling delays, while emission refers to the generation of the final C++ artifacts. Critically, scheduling in Verilator is specifically implemented to simulate the semantic concurrency of SystemVerilog, resolving the execution order of multiple concurrent processes according to language semantics. This represents a distinctive challenge for hardware compilers like Verilator, which must simulate inherently concurrent hardware behavior within a sequential execution model—a key distinction from software compilers, which typically focus on thread-level parallelism and shared-resource management. Errors regarding scheduling and code emission can result in syntactically invalid C++ code or logic that does not accurately reflect the inherently concurrent nature of the hardware simulation cycle. Figure 7 depicts a representative emission error caused by a synchronization failure between the generated C++ header and implementation files. The input Verilog (Figure 7a) contains a `genvar` within a generate loop. Verilator correctly

emitted initialization code for this variable in the C++ implementation file but failed to generate the corresponding declaration in the header file. Consequently, the C++ compiler rejected the output due to an access attempt on an undeclared member. The resolution, shown in Figure 7b, modified the emission filter to explicitly include `genvar` types, ensuring they are correctly declared in the generated class headers.

```
module a;
  for (genvar k = 0; k < 1; k++) begin : gen_empty
  end
endmodule
```

(a) A minimal Verilog example triggering a scheduling/emission error.

```
-    bool refNeedParens = VN_IS(nodep->dtypeSkipRefp(), UnpackArrayDType);
+    const bool refNeedParens = VN_IS(nodep->dtypeSkipRefp(), UnpackArrayDType);

-    if (varp->isIO() || varp->isSignal() || varp->isClassMember() || varp->isTemp()) {
+    if (varp->isIO() || varp->isSignal() || varp->isClassMember() || varp->isTemp() || varp->isGenVar()) {
```

(b) The fix patch of scheduling and emission error.

Fig. 7. An example of a Verilator scheduling and emission error. #6165

Root Cause 3: Type System & Widths. Hardware description languages employ complex type systems involving bit-width inference, signed/unsigned arithmetic, and packed/unpacked arrays. Verilator recorded 65 bugs in this category, where failures arise from incorrect type casting, implicit conversions, or width mismatches. Figure 8 illustrates a representative mixed-sign comparison bug. When comparing a signed 3-bit signal (initialized to binary 111) against an unsigned 4-bit constant (`4'b111`), the IEEE standards dictate zero-extension of the signed operand. However, Verilator incorrectly applied *sign-extension* (becoming 1111), causing the equality check to fail. The fix, shown in Figure 8b, adjusted the compiler's extension logic to ensure signed values are correctly zero-extended when evaluating context-determined expressions involving unsigned operands.

Root Cause 4: Optimization. Verilator applies aggressive transformation passes—such as constant propagation, dead code elimination, and expression simplification—to maximize simulation performance. Optimization errors occur when these transformations are not semantics-preserving, altering the intended behavior of the design. This category accounts for 65 bugs. Figure 9 depicts an optimization fault within the dead code elimination pass. The optimizer incorrectly identified a package as "dead" and removed it from the AST, breaking links to a parameter that referenced a type within that package. The fix, shown in Figure 9b, refined the compiler's dependency tracking. The patch explicitly maps data types to their containing packages and ensures that "liveness" propagates from a data type back to its parent package. This prevents the optimizer from prematurely discarding packages that contain referenced types.

Root Cause 5: Diagnostics & Reporting. This category, containing 24 bugs, covers defects in error localization, misleading warning messages, or the complete failure to report valid errors. Ideally, a robust compiler should guide the user to the source of the problem accurately. Figure 10 shows a case where an array initialization pattern was provided with insufficient elements. Instead of diagnosing the size mismatch, Verilator crashed with an "internal fault." The issue was resolved by implementing a robust assertion mechanism that catches the mismatch and generates a descriptive diagnostic message, as shown in Figure 10b.

Root Cause 6: Syntactic Analysis. Parsing failures account for 23 bugs. These issues occur in the front-end lexer or parser, often due to ambiguity resolution or incorrect handling of grammar rules. Figure 11 highlights a lexer ambiguity where the distribution operator `:/` was followed immediately

```
module a;  // compare signed 3'b111 and unsigned 4'b0111
    reg signed [2:0] s = 3'b111;
    initial case (s)  // signed 3'b111 ----> unsigned 4'b0111
        4'b111: $display("Match Successful.");  // should match
        default: $display("Bug: Match Failed.");
    endcase
endmodule
```

(a) A minimal Verilog example triggering a type system error.

```
    iterateCheck(nodep, "Case expression", nodep->exprp(), CONTEXT_DET, FINAL, subDTypep,
-                EXTEND_LHS);
+                EXTEND_EXP);
```

(b) The fix patch of the type system error.

Fig. 8. An example of a Verilator type system and widths error. #5968

```
package pkg;
    typedef enum { TWO=2 } e;
endpackage

module a;
    localparam p = pkg::TWO;
endmodule
```

(a) A minimal Verilog example triggering an optimization error.

```
+ if (VN_IS(m_modp, Package) || VN_IS(m_modp, Class)) m_dtypePkgsp.emplace(nodep, m_modp);

+ // If data type has a reference in another package, then keep defining package around
+ for (auto& itr : m_dtypePkgsp) {
+     // Propagate usage from type to package
+     if (itr.first->user1()) itr.second->user1Inc();
+ }
```

(b) The fix patch of the optimization error.

Fig. 9. An example of a Verilator optimization error. #5714

```
module a ();
    localparam logic [4-1:0] max_lst [5] = '{0, 1, 2**4-1};
endmodule
```

(a) A minimal Verilog example triggering a diagnostics error.

```
-    iterateConst(nodep->getIndexDefaultedValuep(n));
+    AstNode* const itemp = nodep->getIndexDefaultedValuep(n);
+    UASSERT_OBJ(itemp, nodep, "Missing array init element");
+    iterateConst(itemp);
```

(b) The fix patch of the diagnostics error.

Fig. 10. An example of a Verilator diagnostics and reporting error. #5511

by a value. The tokenizer incorrectly consumed the subsequent character while trying to distinguish the operator from a comment start (//). As depicted in Figure 11b, developers resolved this by modifying the lexer rule to use a lookahead assertion. This adjustment ensures the lexer verifies the context (i.e., that the next character is not a slash) without consuming the character from the input stream.

```
class C;
   rand int x;
   constraint distrib {
      x dist { [1:3] := 0, [5:6], [9:15] :/0 };
   };
endclass
```

(a) A minimal Verilog example triggering a syntactic error.

```
-  ":/"[^\/\*]          { FL; return yP_COLONDIV; }
-  /* : then comment is not ":/" */
+  ":/"/[^\/\*]         { FL; return yP_COLONDIV; }
+  /* : then comment is not ":/" */
```

(b) The fix patch of the syntactic error.

Fig. 11. An example of a Verilator syntactic analysis error. #6121

Root Cause 7: I/O & Environment. There are 23 bugs related to Verilator's interaction with the external environment, including file system operations, include path handling, and OS-specific behaviors. Figure 12 presents a bug where the file reading system command `$fscanf` failed to return the standard EOF code (-1) when the file stream was exhausted, returning 0 instead. This caused parsing loops to hang or behave unpredictably. The corresponding fix in Figure 12b involved adding an explicit check at the function exit to distinguish end-of-file conditions from other states.

```
module a;
  int fd, st, val;
  initial begin
  // a.txt has only one value
    fd = $fopen("a.txt", "r");
    st = $fscanf(fd, "%d", val);
    st = $fscanf(fd, "%d", val);
  // BUG: stop here
    if (st != -1) $error("Error");
  end
endmodule
```

(a) A minimal Verilog example triggering an I/O error.

```
-  for (; pos != format.cend() && !_vl_vsss_eof(fp, floc); ++pos) {
+  for (; pos != format.cend(); ++pos) {
     ...
     done:
+       // Scan stopped early, return parsed or EOF
+       if (_vl_vsss_eof(fp, floc)) return -1;
        return got;
}
```

(b) The fix patch of the EOF handling.

Fig. 12. An example of a Verilator I/O and environment error. #5881

Root Cause 8: Memory Safety. This category encompasses 19 bugs involving segmentation faults, use-after-free errors, and memory leaks within the compiler's runtime. These defects typically manifest as immediate crashes during the compilation process. Figure 13 illustrates a representative heap-use-after-free error. The crash is triggered by a specific concatenation pattern (Figure 13a) during the construction of the compiler's internal Data Flow Graph (DFG). As revealed by the fix in Figure 13b, the error stems from an unsafe evaluation order. In the buggy version, the compiler passes a node pointer (`rhsp`) to the `addDriver` function. This function inadvertently triggers the deallocation of the node's memory. However, the compiler subsequently attempts to access `rhsp->width()` to calculate an offset. Since the memory pointed to by `rhsp` has already been freed, this dereference results in an error. The fix restores memory safety by caching the required width in a local variable before the pointer is passed to the function that invalidates it.

Root Cause 9: Others. The remaining 4 bugs do not fit neatly into the specific categories above, representing rare or idiosyncratic issues that occur infrequently.

Relationship Between Symptoms and Root Causes. We correlate bug symptoms with their root causes to identify which compiler phases drive observable failures, as shown in Figure 14. Our analysis reveals that permissive frontend validation, fragile internal state management, and complex backend translation logic are the primary drivers of Verilator defects.

Specifically, frontend deficiencies are the leading cause of validation failures. The *elaboration and semantic analysis* phase accounts for 81.3% (13/16) of *Accept-Invalid* bugs, where the compiler fails

to reject illegal HDL. While 50% (7/14) of misleading diagnostics stem directly from the reporting logic, the remainder are caused by failures in semantic and type-width analysis. This indicates that frontend permissiveness allows malformed designs to propagate through the pipeline, where they eventually trigger obscure errors or internal instabilities. Furthermore, these frontend weaknesses directly undermine compiler robustness. Crashes are the most frequent symptoms in our dataset, with 36.3% (53/146) occurring during semantic analysis. Additionally, the *Type System and Widths* category accounts for a significant portion of instability, contributing 19.9% (29/146) of total crashes. These figures suggest that Verilator often lacks robust safeguards when encountering complex or unexpected internal states during early design processing.

In contrast, functional miscompilations originate primarily in the backend. Mapping concurrent hardware semantics to sequential C++ remains a significant challenge: *scheduling and code emission* logic drives 75% (48/64) of invalid C++ generation and 41.3% (45/109) of behavioral divergences. Optimization passes further exacerbate this, contributing to 24.8% of divergences. This suggests that backend complexities are the primary threat to simulation correctness. Ultimately, these findings suggest that strengthening early-stage semantic validation and hardening the HDL-to-C++ translation pipeline would most effectively reduce both crashes and miscompilations.

> **Finding 2:** ***Elaboration and Semantics*** *is the dominant root cause, primarily driving frontend failures such as crashes and "accept-invalid" errors due to complexities in hierarchy resolution. Conversely,* ***backend complexity*** *in Scheduling and Emission is the main source of silent functional errors, causing 75% of invalid C++ generation and 41.3% of behavioral divergences. This backend fragility specifically highlights a* ***distinctive challenge*** *Verilator faces: accurately orchestrating inherently concurrent hardware semantics within a sequential execution model. Additionally,* ***optimization passes*** *contributed to 65 bugs by altering design semantics during transformations. These findings reveal a clear dichotomy: frontend bugs lead to instability, while backend and optimization bugs drive miscompilation.*

6 RQ3: Bug Detection

This section examines the characteristics of Verilog and SystemVerilog programs that trigger defects in Verilator. By correlating the structural complexity, language features, and failure modes of these programs, we aim to identify which aspects of the HDL are most error-prone.

First, we calculated and summarized the timeframe of all issues. The median time to the first developer comment is only about 3 hours and the median closed time is 4 days, highlighting developers' rapid reaction and efficient resolution. Unresolved issues, together with issues taking a long time to solve, have the following two characteristics: (1) issues with extremely large codebases, features under active development, or temporarily shelved items; (2) issues impacting Verilator's core architecture, requiring major design changes or having substantial performance implications.

Then we extracted and analyzed the test cases in the issue reports. A typical Verilator test case consists of Verilog or SystemVerilog source code together with a compilation command line; some reports additionally include a C++ testbench used to drive simulation when reporting runtime

```
1  module sim ();
2    wire a, b, c;
3    wire [2:0] c_b_a = {c, b, a};
4  endmodule
```

(a) A minimal Verilog example triggering a memory safety error.

```
+   auto const rhs_width = rhsp->width();
    addDriver(rhsp->fileline(), lsb, rhsp, drivers);
    DfgVertex* const lhsp = concatp->lhsp();
-   addDriver(lhsp->fileline(), lsb + rhsp->width(), lhsp, drivers);
+   addDriver(lhsp->fileline(), lsb + rhs_width, lhsp, drivers);
```

(b) The segmentation fault trace produced by the crash.

Fig. 13. An example of a Verilator memory safety error. #3817

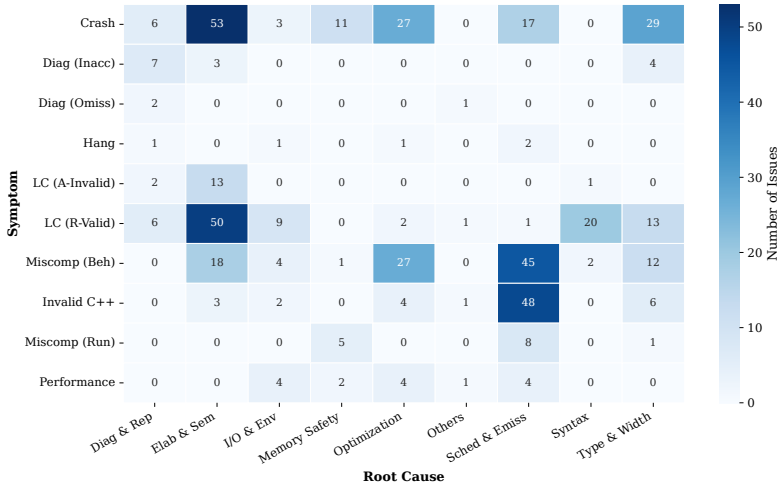


Fig. 14. Heatmap illustrating the correlation between bug symptoms and their underlying root causes.

misbehaviors. In issue reports, these artifacts are usually embedded as Markdown fenced code blocks. Otherwise, they may be provided as file attachments or inline text. In total, we processed the 488 issues and focused on extracting the associated Verilog/SystemVerilog code snippets provided by issue reporters. From these materials, we successfully parsed 338 unique test cases using Verilator's internal parser into Abstract Syntax Trees (ASTs), while the remainder were excluded because they were only described textually, referenced internal unit tests, or relied on specific environment configurations. This set of parsed programs constitutes the corpus used for the analyses that follow.

We next analyze the size and structural complexity of the programs triggering Verilator bugs. Figure 15 and Table 3 present the distribution of size and Abstract Syntax Tree (AST) metrics. Specifically, the median test case is small, containing only **15 Lines of Code (LoC)**, aligning with the short resolution time observed in prior timeframe study. Besides, the mean AST node count is 92.7, resulting in a high semantic density of approximately 4 to 6 AST nodes per line of code. Furthermore, the nesting depth is notably low (Median 2.0). These metrics suggest a clear pattern: Verilator bugs are rarely triggered by deep algorithmic complexity (e.g., deeply nested loops or complex control flow). Instead, they are triggered by shallow but semantically dense interactions between declarative features. For example, a single line declaring a generic class inside a generated block creates a complex internal representation despite low textual volume. This pattern is further mirrored in the developer fixes: the median repair is highly localized, involving only 8 lines of code and typically affecting a single source file. These results imply that while Verilator bugs are often narrow in scope, they cluster within critical semantic analysis phases, suggesting that fuzzing efforts should prioritize the breadth of feature interactions over control-flow depth.

To pinpoint specific error-prone constructs, we traversed the ASTs of the 338 parsed test cases. Table 4 summarizes the prevalence of major Verilog and SystemVerilog constructs. To rigorously examine the relationships between these constructs and the resulting bug causes and symptoms, we conducted a statistical correlation analysis. We applied Fisher's Exact Test [9] with the Benjamini-Hochberg False Discovery Rate (FDR) correction [3] ($p_{adj} < 0.05$) to control for multiple comparisons. Effect sizes were measured using the Log2 Odds Ratio (Log2 OR) [11], utilizing the Haldane-Anscombe correction [1, 12] to account for zero-frequency cells. This analysis revealed three distinct bottlenecks:

Table 3. Structural Complexity of Bug-Triggering Test Cases.

Metric	Mean	Median
Lines of Code (LoC)	22.4	15.0
AST Node Count	92.7	42.0
Max Nesting Depth	1.8	2.0

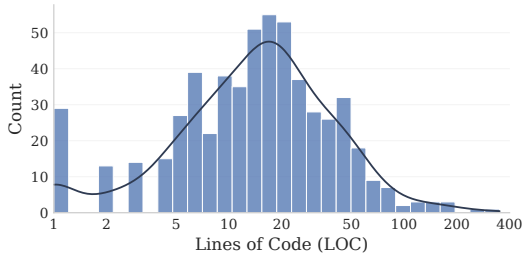


Fig. 15. Distribution of Lines of Code (LoC) in reduced bug-triggering test cases.

Table 4. Prevalence of Verilog/SystemVerilog constructs in bug-triggering tests.

Construct	Count	Prev. (%)	Construct	Count	Prev. (%)
Structural & Procedural			*Data Types & Objects*		
Always (legacy)	116	34.3%	Logic/Reg	302	89.3%
Instantiations	76	22.5%	Class (SV)	93	27.5%
Generate	24	7.1%	Struct/Union	62	18.3%
Always_comb	20	5.9%	Enum	47	13.9%
Always_ff	13	3.8%	Ports	118	34.9%

1. The Persistence of Legacy Semantics: Legacy procedural blocks (`always`) appear in 34.3% of the failing test cases, significantly overshadowing modern, explicitly intended variants like `always_comb` (5.9%) and `always_ff` (3.8%). Because the legacy `always` block is semantically ambiguous, inferring combinational logic, latches or flip-flops based on sensitivity lists, it creates complex edge cases in Verilator's scheduling and graph-ordering logic. Statistically, these blocks are highly correlated with *Optimization* root causes. Since legacy Verilog constructs are robustly supported and typically pass the frontend seamlessly, Verilator applies aggressive backend optimizations to them, which inadvertently exposes subtle defects within the optimization logic.
2. The Complexity of SystemVerilog Abstractions: High-level, software-like abstractions are frequent sources of defects. SystemVerilog `class` constructs appear in 27.5% of cases, and `struct`/`union` constructs in 18.3%. Unlike static hardware signals, these features require dynamic memory management or complex memory layout mapping when translated to C++. Consequently, these modern SystemVerilog features are strongly correlated with *Elaboration & Semantics* root causes. Resolving complex object-oriented scoping, parameterized types, and package imports heavily stresses the frontend elaboration phase. Furthermore, these features are significantly anti-correlated with *Optimization* bugs, suggesting that they often trigger failures early in the elaboration pipeline before they can even reach the backend optimization passes.
3. Translation and Symptom Correlations: Beyond elaboration, `struct` constructs are strongly correlated with *Scheduling & Emission* root causes. Complex structures require intricate flattening to map correctly onto C++ primitives, making the emission phase highly error-prone. Regarding bug symptoms, correlations with code constructs are largely insignificant, with one notable exception: fundamental data types like `logic` and `reg` are strongly anti-correlated with *Diagnostics & Reporting* and *Accept-Invalid* compliance bugs. Because these fundamental data types are heavily tested and robustly handled by the frontend, bugs involving them rarely manifest as trivial parsing or diagnostic errors; instead, they typically surface deeper within the backend execution.

Finding 3: *Bug-triggering test cases are typically small but semantically dense, with failures driven by feature interactions rather than nesting depth. Additionally, software-like abstractions, specifically classes and structs/unions, are frequent error sources, indicating that Verilator's primary bugs lie in managing high-level abstractions.*

7 RQ4: Status of Existing Techniques

Ensuring the reliability of Verilator is a significant challenge for developers, particularly as it evolves to support more complex SystemVerilog constructs while maintaining its performance-driven execution model [10]. Automated testing and verification are essential to catch regressions in this environment. Currently, several testing tools have emerged from both the open-source community and academia, employing diverse strategies for program generation and bug detection. This section evaluates the state-of-the-art in automated Verilog compiler testing.

Table 5 categorizes the selected tools based on their release timeline, generation methodology, and supported features. Community-driven tools, such as VlogHammer [34], often focus on practical utility by generating random combinational circuits to identify discrepancies across various synthesis toolchains. In contrast, academic research has prioritized generating semantically valid and structurally complex test cases. Verismith [14] introduced a grammar-based approach to construct synchronized sequential circuits, specifically avoiding "undefined behavior" (such as `x` or `z` values) to ensure deterministic simulation for differential testing. More recent efforts, such as EvoHDL [33] and VERGEN [17], focus on structural diversity through rule-based generation. The latest advancements shift from pure generation to sophisticated verification and mutation. VeriXmith [35] introduces a mutation-based cross-checking strategy that departs from traditional concrete simulation. By translating compiler outputs into formal SMT-LIB formats, VeriXmith enables formal equivalence checking.

Although most of these tools were originally designed for synthesis engines or general-purpose simulators like Icarus Verilog and Yosys, they can be adapted for Verilator with moderate manual effort. To assess their efficacy in this specific context, we conducted a comparative evaluation focusing on four representative tools: VlogHammer, Verismith, EvoHDL, and VeriXmith. We excluded VERGEN from our evaluation as it is an older technique and its implementation is not publicly available. Our evaluation measured each tool's effectiveness in uncovering bugs specific to Verilator. The experiments were therefore conducted on a single representative version, Verilator v5.002, containing at least 85 known bugs, for three reasons: (1) a static baseline enables controlled assessment of whether existing fuzzing techniques remain effective for mature HDL compiler releases; (2) testing all historical buggy versions over three years is computationally prohibitive; (3) VeriXmith's dependency on KLEE v3.1 limits its support to v5.002, making it the lowest common denominator for fair head-to-head comparison. By standardizing on a single representative version, we ensure a deep, rigorous, and computationally tractable assessment of the existing techniques. For fairness, each tool was allocated 1,000 CPU hours of testing time and executed using the default configurations specified in the respective publications. As summarized in the final column of Table 5, most of these tools identified few or no new Verilator-specific bugs. This performance drop suggests that many previously identified bugs have been patched and that traditional random generation techniques are reaching a point of diminishing returns against modern, mature versions of Verilator. To investigate this trend, we performed a qualitative analysis of the Verilog code generated by each tool. While these tools are proficient at producing syntactically valid code, they often lack the semantic complexity required to trigger Verilator's advanced optimization passes. For instance, although Verismith generates large designs with deep nesting, it relies heavily on basic `always` blocks and simple instantiations, failing to leverage complex SystemVerilog features.

Table 5. Comparison of relevant Verilog testing tools. **Gen. Strategy:** Approach used to create or modify code. **Oracle:** Mechanism used to determine correctness. **Scope:** ● = Full, ◐ = Partial, ○ = None.

Tool	Year	Gen. Strategy	Oracle Type	Language Scope			#Bugs	
				SV	Seq.	Synth.	Reported[1]	Tested[2]
VERGEN	2003	Grammar-based	Differential	○	●	●	0 (0)	-
VlogHammer	2013	Grammar-based	Differential	○	○	●	22 (66)	0
Verismith	2020	Grammar-based	Differential	○	●	●	0 (11)	0
EvoHDL	2025	Grammar-based	Differential	◐	●	●	0 (20)	0
VeriXmith	2025	Mutation-based	Metamorphic	●	●	●	1 (31)	1

[1] Values outside and inside brackets represent Verilator-specific and total developer-reported bugs, respectively.
[2] Values in this column represent bugs detected in our experiments.
Key: SV = SystemVerilog; Seq. = Sequential Logic; Synth. = Synthesizable.

These findings highlight a critical need for specialized testing methodologies that are specifically tailored to Verilator's unique architecture and performance-driven execution model.

> **Finding 4:** *Our evaluation of state-of-the-art testing tools against Verilator reveals diminishing returns in bug discovery, with most tools identifying few or no new defects. This also suggests a* ***semantic gap****: current generators produce syntactically valid code but lack the complexity to trigger advanced optimization passes or exercise sophisticated SystemVerilog constructs. These results highlight that existing automated methodologies are currently insufficient for uncovering the subtle functional errors prevalent in modern compiler releases.*

8 Implications and Discussion

8.1 Actionable Suggestions

Recommendations for Developers. To enhance Verilator's reliability, developers are recommended to prioritize early semantic validation. Since 81.3% of "accept-invalid" bugs arise during elaboration, stricter frontend checks can prevent malformed states from propagating into backend crashes. Furthermore, they should also strengthen the *translation pipeline*, particularly the scheduling logic that maps concurrent hardware semantics to sequential C++. As Scheduling & Emission is the second-largest root-cause category, testing should target same-cycle evaluation order and unintended combinational loops. Finally, legacy and **high-level constructs** require dedicated hardening: *always* blocks need stress tests for ambiguous sensitivity lists, while Classes and Unions warrant regression suites focused on memory management and data layout.

Future Directions for Researchers. While state-of-the-art tools (e.g., Verismith) have successfully advanced beyond basic syntactic validity, significant opportunities remain to enhance the **semantic diversity and logical depth**. Future research should move beyond syntactic validity to generate "shallow but dense" programs designed to exercise complex optimization passes like dead code elimination. Additionally, there is a critical need for generators targeting **advanced SystemVerilog features**, such as object-oriented constructs, which remain under-tested by current methodologies. Finally, as miscompilation is the dominant symptom, researchers must develop **robust test oracles** beyond crash detection, focusing on identifying subtle behavioral divergences in generated simulation models.

8.2 Threats to Validity

Internal Validity. To mitigate selection bias, we followed established compiler study methodologies [29] by restricting our dataset to fixed bugs with patches. While this excludes unverified reports, it ensures a high-fidelity dataset that captures the critical transition to Verilator.

Construct Validity. To distinguish Verilator faults from downstream toolchain errors, we strictly excluded external C++ compiler bugs. For subjectivity in manual classification, two authors independently labeled bugs using a rigorous taxonomy, with a third author resolving disagreements.

External Validity. While focusing on Verilator limits direct generalizability to other simulators, our analysis targets fundamental compilation stages like elaboration and scheduling which are universal to HDL tools. Consequently, the identified patterns of miscompilation likely reflect broader challenges within the hardware verification ecosystem. However, we cannot determine the exact distribution between fuzzer-generated and organically discovered reports, which limits the generalizability of our findings to real-world fuzzing practices. Furthermore, our evaluation of the baseline fuzzers (RQ4) was conducted on a single, static version (Verilator v5.002) rather than the entire historical dataset of buggy versions, which inherently excludes the evaluation of regression bugs and may not fully capture the fuzzers' effectiveness on other releases of Verilator. Therefore, our claims regarding testing tool efficacy apply within this specific baseline context.

9 Related Work

Empirical Studies on Compiler Bugs. Prior work has characterized bugs in C/C++ compilers, including variations in bug lifespan and priority [29] and optimization errors caused by faulty code transformations [36]. Related studies have examined JVM type-system bugs [4] and defects in WebAssembly [19], Solidity [16], Rust [15], and deep learning compilers [22]. Our study reveals both shared and domain-specific patterns. Like GCC and LLVM, Verilator frequently exhibits crashes triggered by small, localized inputs. However, translating concurrent, cycle-accurate SystemVerilog designs into sequential C++ simulations creates distinctive failures, including *Scheduling & Emission* bugs caused by flattening concurrent semantics and frequent "Reject-Valid" failures involving complex object-oriented constructs. By distinguishing common compiler weaknesses from EDA-specific ones, our taxonomy informs reliability improvements for hardware simulation toolchains.

HDL Toolchain Testing. Testing for HDL tools mainly focused on synthesis tools, which convert code into hardware gates. For example, Herklotz et al. [14] developed Verismith, a fuzzing framework for synthesis tools. This was later enhanced by EvoHDL [33], which uses evolutionary techniques to generate complex logic structures capable of triggering deep logic errors in commercial toolchains. Recent work has also targeted simulators through constraint-solver-based differential checking [35]. These studies propose bug-finding techniques but do not systematically characterize the bugs they expose. We provide the first comprehensive empirical study of Verilator bugs, identifying prevalent patterns and root causes to support more targeted testing.

10 Conclusion

This paper presents the first empirical study of Verilator, revealing that crashes and silent *miscompilations* are the dominant threat to simulation integrity. We identify a clear architectural dichotomy: while frontend elaboration issues typically manifest as crashes, backend scheduling phases are the primary source of these silent, valid-but-incorrect code generations. Furthermore, we find that bugs are triggered not by code volume, but by the *semantic density* of interactions between high-level abstractions and legacy hardware constructs. Current testing tools, however, exhibit a significant "semantic gap", failing to generate the complexity required to exercise these error-prone optimization passes. Consequently, we advocate for a shift toward semantics-aware fuzzing to bridge this gap and ensure the robustness of critical HDL compilation infrastructure.

Data Availability

The artifact related to this paper is publicly available at: https://doi.org/10.5281/zenodo.18424608.

Acknowledgments

We thank the anonymous reviewers for their valuable feedback. This work was supported in part by the National Natural Science Foundation of China (Grants 624B2067, 62572226, and 62472215), the Jiangsu Natural Science Foundation (Grant BK20231402), the NJU-Huawei Joint Innovation Lab, the Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China (No. JYB2025XDXM118), the "111 Center" (No. B26023), and the Collaborative Innovation Center of Novel Software Technology and Industrialization.

References

[1] Frank J Anscombe. 1956. On estimating binomial response relations. *Biometrika* 43, 3/4 (1956), 461–464. doi:10.1093/biomet/43.3-4.461

[2] Scott Beamer. 2020. A Case for Accelerating Software RTL Simulation. *IEEE Micro* 40, 4 (2020), 112–119. doi:10.1109/MM.2020.2997639

[3] Yoav Benjamini and Yosef Hochberg. 1995. Controlling the false discovery rate: a practical and powerful approach to multiple testing. *Journal of the Royal statistical society: series B (Methodological)* 57, 1 (1995), 289–300. doi:10.1111/j.2517-6161.1995.tb02031.x

[4] Stefanos Chaliasos, Thodoris Sotiropoulos, Georgios-Petros Drosos, Charalambos Mitropoulos, Dimitris Mitropoulos, and Diomidis Spinellis. 2021. Well-typed programs can go wrong: a study of typing-related bugs in JVM compilers. *Proc. ACM Program. Lang.* 5, OOPSLA (2021), 1–30. doi:10.1145/3485500

[5] Qinlin Chen, Nairen Zhang, Jinpeng Wang, Tian Tan, Chang Xu, Xiaoxing Ma, and Yue Li. 2023. The Essence of Verilog: A Tractable and Tested Operational Semantics for Verilog. *Proc. ACM Program. Lang.* 7, OOPSLA2, Article 230 (Oct. 2023), 30 pages. doi:10.1145/3622805

[6] Yuan Chi, Xian Lin, and Xin Zheng. 2023. Design of High-performance SoC Simulation Model Based on Verilator. In *Proceedings of the 2022 5th International Conference on Algorithms, Computing and Artificial Intelligence* (Sanya, China) *(ACAI '22)*. Association for Computing Machinery, New York, NY, USA, Article 92, 6 pages. doi:10.1145/3579654.3579751

[7] Alastair F. Donaldson, Dilan Sheth, Jean-Baptiste Tristan, and Alex Usher. 2024. Randomised Testing of the Compiler for a Verification-Aware Programming Language. In *IEEE Conference on Software Testing, Verification and Validation, ICST 2024, Toronto, ON, Canada, May 27-31, 2024*. IEEE, 407–418. doi:10.1109/ICST60714.2024.00044

[8] Fares Elsabbagh, Shabnam Sheikhha, Victor A. Ying, Quan M. Nguyen, Joel S. Emer, and Daniel Sánchez. 2023. Accelerating RTL Simulation with Hardware-Software Co-Design. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2023, Toronto, ON, Canada, 28 October 2023 - 1 November 2023*. ACM, 153–166. doi:10.1145/3613424.3614257

[9] Ronald A Fisher. 1922. On the interpretation of χ 2 from contingency tables, and the calculation of P. *Journal of the royal statistical society* 85, 1 (1922), 87–94. doi:10.1111/j.2397-2335.1922.tb00768.x

[10] Peter Flake, Phil Moorby, Steve Golson, Arturo Salz, and Simon Davidmann. 2020. Verilog HDL and its ancestors and descendants. *Proc. ACM Program. Lang.* 4, HOPL, Article 87 (June 2020), 90 pages. doi:10.1145/3386337

[11] Joseph L Fleiss, Bruce Levin, and Myunghee Cho Paik. 2003. *Statistical methods for rates and proportions*. john wiley & sons. doi:10.1002/0471445428

[12] J. B. S. HALDANE. 1956. THE ESTIMATION AND SIGNIFICANCE OF THE LOGARITHM OF A RATIO OF FREQUENCIES. *Annals of Human Genetics* 20, 4 (1956), 309–311. doi:10.1111/j.1469-1809.1955.tb01285.x

[13] Eunkyung Ham, Yujin Jeon, Jaeyun Lim, and Ji-Hoon Kim. 2023. Verilator-based Fast Verification Methodology for BLE MAC Hardware. In *International Conference on Electronics, Information, and Communication, ICEIC 2023, Singapore, February 5-8, 2023*. IEEE, 1–3. doi:10.1109/ICEIC57457.2023.10049940

[14] Yann Herklotz and John Wickerson. 2020. Finding and Understanding Bugs in FPGA Synthesis Tools. In *Proceedings of the 2020 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays* (Seaside, CA, USA) *(FPGA '20)*. Association for Computing Machinery, New York, NY, USA, 277–287. doi:10.1145/3373087.3375310

[15] Zixi Liu, Yang Feng, Yunbo Ni, Shaohua Li, Xizhe Yin, Qingkai Shi, Baowen Xu, and Zhendong Su. 2025. An Empirical Study of Bugs in the rustc Compiler. *Proc. ACM Program. Lang.* 9, OOPSLA2, Article 411 (Oct. 2025), 28 pages. doi:10.1145/3763800

[16] Haoyang Ma, Wuqi Zhang, Qingchao Shen, Yongqiang Tian, Junjie Chen, and Shing-Chi Cheung. 2024. Towards Understanding the Bugs in Solidity Compiler. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis* (Vienna, Austria) *(ISSTA 2024)*. Association for Computing Machinery, New York, NY, USA, 1312–1324. doi:10.1145/3650212.3680362

[17] Boris Ratchev, Mike Hutton, Gregg Baeckler, and Babette van Antwerpen. 2003. Verifying the correctness of FPGA logic synthesis algorithms. In *Proceedings of the 2003 ACM/SIGDA Eleventh International Symposium on Field Programmable*

Gate Arrays (Monterey, California, USA) *(FPGA '03)*. Association for Computing Machinery, New York, NY, USA, 127–135. doi:10.1145/611817.611837

[18] David I. Rich. 2003. The Evolution of SystemVerilog. *IEEE Design & Test of Computers* 20, 4 (2003), 82–84. doi:10.1109/MDT.2003.1214355

[19] Alan Romano, Xinyue Liu, Yonghwi Kwon, and Weihang Wang. 2021. An Empirical Study of Bugs in WebAssembly Compilers. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 42–54. doi:10.1109/ASE51524.2021.9678776

[20] Margrit Schreier. 2012. Qualitative content analysis in practice. (2012). doi:10.4135/9781529682571

[21] Carolyn B. Seaman. 1999. Qualitative Methods in Empirical Studies of Software Engineering. *IEEE Trans. Software Eng.* 25, 4 (1999), 557–572. doi:10.1109/32.799955

[22] Qingchao Shen, Haoyang Ma, Junjie Chen, Yongqiang Tian, Shing-Chi Cheung, and Xiang Chen. 2021. A comprehensive study of deep learning compiler bugs. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (Athens, Greece) *(ESEC/FSE 2021)*. Association for Computing Machinery, New York, NY, USA, 968–980. doi:10.1145/3468264.3468591

[23] Wilson Snyder. 2004. Verilator and systemperl. In *North American SystemC Users' Group, Design Automation Conference*, Vol. 79. 122–148. https://veripool.org/papers/verilator_systemperl_nascug.pdf

[24] Wilson Snyder. 2010. Verilator: Fast, Free, But for Me? https://veripool.org/papers/Verilator_Fast_Free_Me_DVClub10_pres.pdf Retrieved: January 24, 2026.

[25] Wilson Snyder. 2018. Verilator 4.0: Open Simulation Goes Multithreaded. https://www.veripool.com/papers/Verilator_v4_Multithreaded_OrConf2018.pdf Retrieved: January 24, 2026.

[26] Wilson Snyder. 2020. Verilator Internals 1: Internal structures and debugging of Verilator. Presentation slides. https://veripool.org/papers/Verilator_Internals1_202010.pdf Accessed: 2026-01-29.

[27] Wilson Snyder. 2024. Verilator. https://www.veripool.org/verilator/ Retrieved: January 30, 2025.

[28] Wilson Snyder. 2026. *Verilator User's Guide*. Veripool. https://www.veripool.org/ftp/verilator_doc.pdf Version 5.045.

[29] Chengnian Sun, Vu Le, Qirun Zhang, and Zhendong Su. 2016. Toward understanding compiler bugs in GCC and LLVM. In *Proceedings of the 25th International Symposium on Software Testing and Analysis* (Saarbrücken, Germany) *(ISSTA 2016)*. Association for Computing Machinery, New York, NY, USA, 294–305. doi:10.1145/2931037.2931074

[30] Laung-Terng Wang, Yao-Wen Chang, and Kwang-Ting Tim Cheng. 2009. *Electronic design automation: synthesis, verification, and test*. Morgan Kaufmann. doi:10.1016/S1875-9661(08)X0006-4

[31] Stephen Williams. 2025. Icarus Verilog. https://steveicarus.github.io/iverilog/ Retrieved: January 24, 2026.

[32] Stephen Williams and Michael Baxter. 2002. Icarus verilog: open-source verilog more than a year later. *Linux J.* 2002, 99 (July 2002), 3. https://www.linuxjournal.com/article/6001

[33] Zhihao Xu, Shikai Guo, Guilin Zhao, Peiyu Zou, Xiaochen Li, and He Jiang. 2025. A Novel HDL Code Generator for Effectively Testing FPGA Logic Synthesis Compilers. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 44, 11 (2025), 4405–4418. doi:10.1109/TCAD.2025.3565488

[34] Yosys HQ. 2013. VlogHammer. https://yosyshq.net/yosys/vloghammer.html. Accessed: 2025-12-04.

[35] Yike Zhou, Yanyan Jiang, and Jian Lu. 2025. Unveiling Cross-checking Opportunities in Verilog Compilers. *ACM Trans. Design Autom. Electr. Syst.* 30, 2 (2025), 32:1–32:23. doi:10.1145/3715325

[36] Zhide Zhou, Zhilei Ren, Guojun Gao, and He Jiang. 2021. An empirical study of optimization bugs in GCC and LLVM. *Journal of Systems and Software* 174 (2021), 110884. doi:10.1016/j.jss.2020.110884

Received 2026-01-30; accepted 2026-06-25